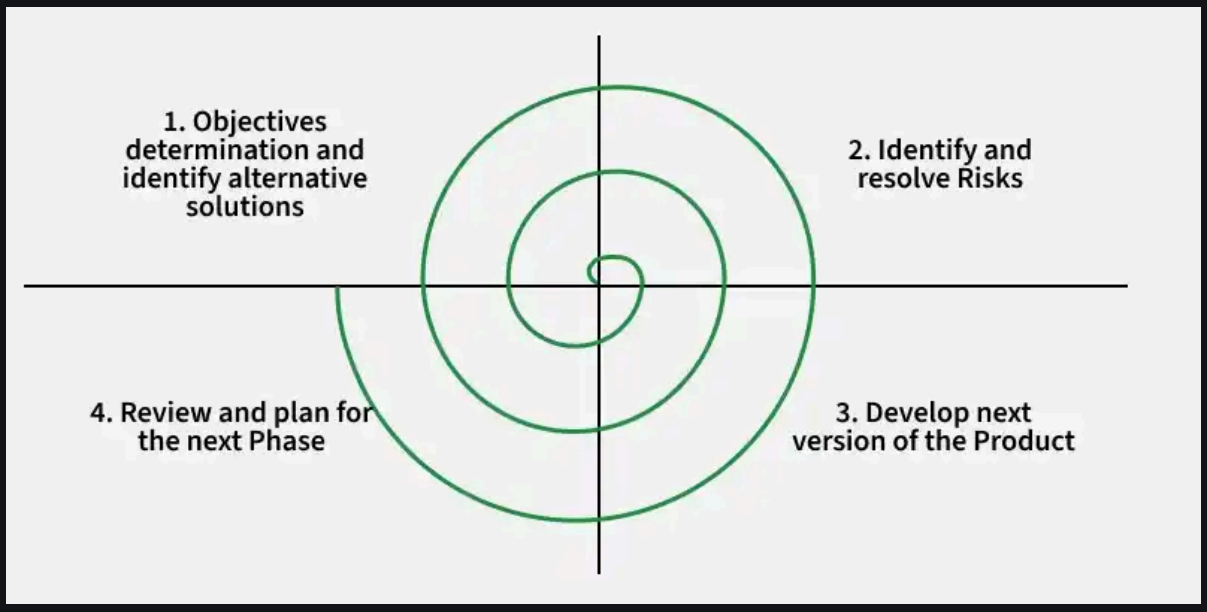
Spiral Model in Software Engineering

Last Updated : 13 Jul, 2026

The spiral model is a risk-driven software development process model. Based on the unique risk patterns of a given project, the spiral model guides a team to adopt elements of one or more process models, such as incremental, waterfall, or evolutionary prototyping.



Phases of Spiral Model

Phases of the Spiral Model

Focus is on managing risk through multiple iterations of the software development process. Each phase of the Spiral Model is divided into four Quadrants.

1. Objective Setting (Planning)

- Project goals and requirements are identified
- Functional and non-functional requirements are analyzed
- Possible solutions are explored

2. Risk Analysis

- Risks related to cost, schedule, performance, and technology are identified
- Best solution is selected
- Prototypes are built to reduce risks

3. Development and Testing

- Selected features are designed, developed, and tested

- The next working version of the software is created

4. Review and Planning

- Customers evaluate the current version
- Feedback is collected
- Planning for the next iteration begins

The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation. The Radius of the spiral at any point represents the expenses (cost) of the project so far, and the angular dimension represents the progress made so far in the current phase.

Risk Handling

- Risks are identified and analyzed at the beginning
- Prototypes help detect and resolve risks early
- Continuous risk evaluation at every iteration
- More flexible than Prototyping and Waterfall models

> *The Spiral Model is often called a Meta-Model because it combines concepts from multiple SDLC models such as Waterfall, Iterative, and Prototyping models.*

Example

Following shows how the Spiral Model supports continuous improvement and risk management at every stage in building E Commerce Website.

- **Requirements & Prototype:** Gather basic requirements like product listing, cart, and payment, identify risks, and build a prototype.
- **Design & Development:** Refine the design using feedback and develop core features such as secure payments, user registration, and shopping cart functionality, then test and refine them.
- **Advanced Features:** Add advanced features such as order tracking, product reviews, and search functionality are developed, and also test performance, security, and scalability.
- **Deployment & Maintenance:** The system is fully tested, deployed for users, and continuously monitored to address remaining risks and make further improvements.

Advantages

- **Effective Risk Handling:** Identifies and resolves risks at every phase, making it ideal for projects with high uncertainty.
- **Suitable for Large Projects:** Best suited for large and complex software systems.
- **Flexible Requirements:** Allows changes in requirements even at later stages of development.
- **Early Customer Feedback:** Customers can review early versions of the product, improving satisfaction.
- **Iterative Development:** Software is developed in repeated cycles, allowing gradual improvement.
- **Better Communication:** Regular reviews improve coordination between customers and developers.
- **High Software Quality:** Continuous testing and refinement lead to reliable and high-quality software.

Disadvantages

- **Complex to Manage:** The model is complex due to multiple iterations and continuous risk analysis.
- **High Cost:** Not suitable for small projects because it requires more time, effort, and resources.
- **Depends Heavily on Risk Analysis:** Success depends on accurate risk identification, which requires experienced professionals.
- **Difficult Time Estimation:** The number of iterations is not fixed, making scheduling difficult.
- **Time-Consuming:** Frequent reviews and evaluations can increase development time.
- **Resource Intensive:** Requires significant investment in planning, risk management, and testing.

Suggested Quiz

The Spiral Model is mainly used for:

A Small projects with fixed requirements

B Rapid coding without planning

C Risk handling in large and complex projects

D Projects with no customer involvement

Report Issue Login to View Explanation

1/4 < Previous Next >

Comment GeeksforGeeks 330

Explore